Modelo Vista Controlador (MVC) en PHP

Introducción al Patrón MVC

El Modelo Vista Controlador (MVC) es un patrón de arquitectura de software que separa una aplicación en tres componentes principales, cada uno con responsabilidades específicas. Este patrón facilita la organización del código, mejora su mantenibilidad y permite a múltiples desarrolladores trabajar simultáneamente en diferentes aspectos de la aplicación.

¿Por qué utilizar MVC?

Antes de profundizar en los detalles del patrón MVC, es importante entender por qué se ha convertido en un estándar en el desarrollo web moderno:

- **Separación de responsabilidades**: Cada componente tiene un propósito específico
- **Código más organizado**: Facilita encontrar y corregir errores
- **Mantenimiento simplificado**: Cambios en un componente no afectan necesariamente a otros
- **Facilita el trabajo en equipo**: Diferentes desarrolladores pueden trabajar en distintos componentes
- **Reutilización de código**: Los componentes pueden utilizarse en diferentes partes de la aplicación
- **Mejor para aplicaciones complejas**: A medida que la aplicación crece, mantiene la organización

Los Tres Componentes de MVC

Modelo

El Modelo representa la lógica de negocio y los datos de la aplicación. Es responsable de:

- Acceder a la base de datos
- Definir las reglas de negocio
- Procesar y manipular datos
- Notificar a la Vista de cambios en los datos (en implementaciones avanzadas)

El Modelo **no debe** preocuparse por cómo se presentan los datos ni cómo el usuario interactúa con ellos.

Ejemplo de un Modelo en PHP

```php
<?php
class ProductModel {
    private $db; // Conexión a la base de datos

    public function __construct($database) {
        $this->db = $database;
    }

    // Obtener todos los productos
    public function getAllProducts() {
        $query = "SELECT * FROM products";
```

```
        $stmt = $this->db->prepare($query);
        $stmt->execute();

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    // Obtener un producto por ID
    public function getProductById($id) {
        $query = "SELECT * FROM products WHERE id = :id";
        $stmt = $this->db->prepare($query);
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->fetch(PDO::FETCH_ASSOC);
    }

    // Crear un nuevo producto
    public function createProduct($name, $price, $description) {
        $query = "INSERT INTO products (name, price, description) VALUES
(:name, :price, :description)";
        $stmt = $this->db->prepare($query);
        $stmt->bindParam(':name', $name, PDO::PARAM_STR);
        $stmt->bindParam(':price', $price, PDO::PARAM_STR);
        $stmt->bindParam(':description', $description, PDO::PARAM_STR);

        return $stmt->execute();
    }
}
?>
```

Vista

La Vista es responsable de presentar los datos al usuario. Se encarga de:

- Mostrar la información al usuario
- Presentar interfaces de usuario
- Recibir datos del Modelo (usualmente a través del Controlador)
- Contener elementos visuales y de interfaz

La Vista **no debe** procesar datos ni implementar lógica de negocio.

Ejemplo de una Vista en PHP

```
<?php
// product_list.php - Vista para mostrar la lista de productos
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
```

```
    <title>Lista de Productos</title>
    <link rel="stylesheet" href="assets/css/styles.css">
</head>
<body>
    <div class="container">
        <h1>Catálogo de Productos</h1>

        <?php if (empty($products)): ?>
            <p>No hay productos disponibles.</p>
        <?php else: ?>
            <div class="product-grid">
                <?php foreach ($products as $product): ?>
                    <div class="product-card">
                        <h2><?= htmlspecialchars($product['name']) ?></h2>
                        <p class="price">$<?=
htmlspecialchars($product['price']) ?></p>
                        <p class="description"><?=
htmlspecialchars($product['description']) ?></p>
                        <a href="index.php?action=view&id=<?= $product['id'] ?
>" class="btn">Ver detalles</a>
                    </div>
                <?php endforeach; ?>
            </div>
        <?php endif; ?>
    </div>
</body>
</html>
```

Controlador

El Controlador actúa como intermediario entre el Modelo y la Vista. Es responsable de:

- Recibir las solicitudes del usuario
- Procesar la entrada del usuario
- Interactuar con el Modelo para obtener o modificar datos
- Seleccionar qué Vista mostrar al usuario
- Pasar datos del Modelo a la Vista

El Controlador **no debe** contener lógica de negocio compleja ni código de presentación.

Ejemplo de un Controlador en PHP

```
<?php
class ProductController {
    private $model;

    public function __construct($database) {
        // Inicializar el modelo
        $this->model = new ProductModel($database);
    }
```

```
    public function index() {
        // Obtener todos los productos del modelo
        $products = $this->model->getAllProducts();

        // Incluir la vista
        include 'views/product_list.php';
    }

    public function view($id) {
        // Obtener un producto específico
        $product = $this->model->getProductById($id);

        if ($product) {
            // Si el producto existe, mostrar la vista de detalles
            include 'views/product_detail.php';
        } else {
            // Si no existe, mostrar error
            include 'views/error.php';
        }
    }

    public function create() {
        // Mostrar formulario de creación
        if ($_SERVER['REQUEST_METHOD'] === 'GET') {
            include 'views/product_form.php';
        }
        // Procesar datos del formulario
        else if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $name = $_POST['name'] ?? '';
            $price = $_POST['price'] ?? 0;
            $description = $_POST['description'] ?? '';

            if ($this->model->createProduct($name, $price, $description)) {
                // Redirigir a la lista después de crear
                header('Location: index.php?action=index');
                exit;
            } else {
                // Mostrar error si no se pudo crear
                $error = "No se pudo crear el producto";
                include 'views/product_form.php';
            }
        }
    }
}
?>
```

Flujo de Trabajo en MVC

El flujo típico de una solicitud en una aplicación MVC es el siguiente:

```
    U Cómo Usuario
    C Cómo Controlador
    M Cómo Modelo
    V Cómo Vista

    U->>C: Solicita una página
    C->>M: Solicita datos
    M->>C: Devuelve datos
    C->>V: Envía datos para mostrar
    V->>U: Muestra la página al usuario
```

1. El usuario realiza una acción (ej. visita una URL)
2. El controlador recibe la solicitud
3. El controlador interactúa con el modelo si es necesario
4. El modelo devuelve datos al controlador
5. El controlador selecciona una vista y le pasa los datos
6. La vista renderiza la interfaz que ve el usuario

Implementación Básica de MVC en PHP

A continuación, veremos cómo implementar un sistema MVC básico en PHP sin utilizar un framework.

Estructura de Directorios

Una estructura de directorios típica para un proyecto MVC podría ser:

```
proyecto/
├── config/
│   └── database.php
├── controllers/
│   ├── ProductController.php
│   └── UserController.php
├── models/
│   ├── ProductModel.php
│   └── UserModel.php
├── views/
│   ├── products/
│   │   ├── list.php
│   │   ├── detail.php
│   │   └── form.php
│   └── users/
│       ├── list.php
│       └── form.php
├── public/
│   ├── index.php
│   ├── css/
│   └── js/
└── utils/
    └── Router.php
```

Archivo de Punto de Entrada (index.php)

```php
<?php
// public/index.php - Punto de entrada de la aplicación

// Incluir archivos de configuración
require_once '../config/database.php';

// Autocargar clases
spl_autoload_register(function ($className) {
    // Convertir namespace a ruta de archivo
    $file = str_replace('\\', DIRECTORY_SEPARATOR, $className) . '.php';

    // Buscar en directorios de la aplicación
    $directories = ['controllers', 'models', 'utils'];
    foreach ($directories as $directory) {
        $path = "../{$directory}/{$file}";
        if (file_exists($path)) {
            require_once $path;
            return;
        }
    }
});

// Crear conexión a la base de datos
$db = new PDO(
    "mysql:host={$dbConfig['host']};dbname={$dbConfig['dbname']};charset=utf8",
    $dbConfig['username'],
    $dbConfig['password']
);
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

// Obtener acción y controlador de la URL
$controller = $_GET['controller'] ?? 'product';
$action = $_GET['action'] ?? 'index';
$id = $_GET['id'] ?? null;

// Mapear el controlador solicitado a la clase correspondiente
$controllerMap = [
    'product' => 'ProductController',
    'user' => 'UserController'
];

// Verificar si existe el controlador
if (!isset($controllerMap[$controller])) {
    header("HTTP/1.0 404 Not Found");
    echo "Controlador no encontrado";
    exit;
}
```

```php
// Crear instancia del controlador
$controllerClass = $controllerMap[$controller];
$controllerInstance = new $controllerClass($db);

// Verificar si existe el método/acción
if (!method_exists($controllerInstance, $action)) {
    header("HTTP/1.0 404 Not Found");
    echo "Acción no encontrada";
    exit;
}

// Ejecutar la acción
if ($id !== null) {
    $controllerInstance->$action($id);
} else {
    $controllerInstance->$action();
}
?>
```

Conexión a la Base de Datos (database.php)

```php
<?php
// config/database.php

$dbConfig = [
    'host' => 'localhost',
    'dbname' => 'tienda_virtual',
    'username' => 'usuario',
    'password' => 'contraseña'
];
?>
```

Ejemplo Completo: Gestión de Productos

Veamos un ejemplo completo de cómo implementar un sistema CRUD (Crear, Leer, Actualizar, Eliminar) para productos utilizando MVC.

Modelo de Producto (ProductModel.php)

```php
<?php
// models/ProductModel.php

class ProductModel {
    private $db;

    public function __construct($database) {
        $this->db = $database;
```

```
    }

    // Obtener todos los productos
    public function getAllProducts() {
        $query = "SELECT * FROM products ORDER BY name";
        $stmt = $this->db->prepare($query);
        $stmt->execute();

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    // Obtener un producto por ID
    public function getProductById($id) {
        $query = "SELECT * FROM products WHERE id = :id";
        $stmt = $this->db->prepare($query);
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->fetch(PDO::FETCH_ASSOC);
    }

    // Crear un nuevo producto
    public function createProduct($data) {
        $query = "INSERT INTO products (name, price, description, stock)
                  VALUES (:name, :price, :description, :stock)";
        $stmt = $this->db->prepare($query);

        return $stmt->execute([
            ':name' => $data['name'],
            ':price' => $data['price'],
            ':description' => $data['description'],
            ':stock' => $data['stock']
        ]);
    }

    // Actualizar un producto existente
    public function updateProduct($id, $data) {
        $query = "UPDATE products
                  SET name = :name,
                      price = :price,
                      description = :description,
                      stock = :stock
                  WHERE id = :id";

        $stmt = $this->db->prepare($query);

        return $stmt->execute([
            ':id' => $id,
            ':name' => $data['name'],
            ':price' => $data['price'],
            ':description' => $data['description'],
            ':stock' => $data['stock']
        ]);
```

```
    }

    // Eliminar un producto
    public function deleteProduct($id) {
        $query = "DELETE FROM products WHERE id = :id";
        $stmt = $this->db->prepare($query);
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);

        return $stmt->execute();
    }
}
?>
```

Controlador de Producto (ProductController.php)

```
<?php
// controllers/ProductController.php

class ProductController {
    private $model;

    public function __construct($database) {
        $this->model = new ProductModel($database);
    }

    // Listar todos los productos
    public function index() {
        $products = $this->model->getAllProducts();
        include '../views/products/list.php';
    }

    // Ver detalle de un producto
    public function view($id) {
        $product = $this->model->getProductById($id);

        if ($product) {
            include '../views/products/detail.php';
        } else {
            $error = "Producto no encontrado";
            include '../views/error.php';
        }
    }

    // Crear un nuevo producto
    public function create() {
        // Mostrar formulario
        if ($_SERVER['REQUEST_METHOD'] === 'GET') {
            include '../views/products/form.php';
        }
        // Procesar formulario
```

```
        else if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = [
                'name' => $_POST['name'] ?? '',
                'price' => $_POST['price'] ?? 0,
                'description' => $_POST['description'] ?? '',
                'stock' => $_POST['stock'] ?? 0
            ];

            // Validación básica
            $errors = [];
            if (empty($data['name'])) {
                $errors[] = "El nombre es obligatorio";
            }
            if (!is_numeric($data['price']) || $data['price'] <= 0) {
                $errors[] = "El precio debe ser un número positivo";
            }

            if (empty($errors)) {
                if ($this->model->createProduct($data)) {
                    header('Location: index.php?
controller=product&action=index');
                    exit;
                } else {
                    $error = "Error al crear el producto";
                    include '../views/products/form.php';
                }
            } else {
                include '../views/products/form.php';
            }
        }
    }

    // Actualizar un producto existente
    public function edit($id) {
        $product = $this->model->getProductById($id);

        if (!$product) {
            $error = "Producto no encontrado";
            include '../views/error.php';
            return;
        }

        // Mostrar formulario con datos actuales
        if ($_SERVER['REQUEST_METHOD'] === 'GET') {
            include '../views/products/form.php';
        }
        // Procesar actualización
        else if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $data = [
                'name' => $_POST['name'] ?? '',
                'price' => $_POST['price'] ?? 0,
                'description' => $_POST['description'] ?? '',
                'stock' => $_POST['stock'] ?? 0
```

```
            ];

            // Validación básica
            $errors = [];
            if (empty($data['name'])) {
                $errors[] = "El nombre es obligatorio";
            }
            if (!is_numeric($data['price']) || $data['price'] <= 0) {
                $errors[] = "El precio debe ser un número positivo";
            }

            if (empty($errors)) {
                if ($this->model->updateProduct($id, $data)) {
                    header('Location: index.php?
controller=product&action=index');
                    exit;
                } else {
                    $error = "Error al actualizar el producto";
                    include '../views/products/form.php';
                }
            } else {
                include '../views/products/form.php';
            }
        }
    }

    // Eliminar un producto
    public function delete($id) {
        $product = $this->model->getProductById($id);

        if (!$product) {
            $error = "Producto no encontrado";
            include '../views/error.php';
            return;
        }

        // Mostrar confirmación
        if ($_SERVER['REQUEST_METHOD'] === 'GET') {
            include '../views/products/delete.php';
        }
        // Procesar eliminación
        else if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            if ($this->model->deleteProduct($id)) {
                header('Location: index.php?controller=product&action=index');
                exit;
            } else {
                $error = "Error al eliminar el producto";
                include '../views/products/delete.php';
            }
        }
    }
}
?>
```

Vista de Lista de Productos (list.php)

```
<?php
// views/products/list.php
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Lista de Productos</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>Catálogo de Productos</h1>
            <a href="index.php?controller=product&action=create" class="btn
btn-primary">Nuevo Producto</a>
        </header>

        <?php if (empty($products)): ?>
            <p class="alert alert-info">No hay productos disponibles.</p>
        <?php else: ?>
            <table class="table">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Nombre</th>
                        <th>Precio</th>
                        <th>Stock</th>
                        <th>Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($products as $product): ?>
                        <tr>
                            <td><?= htmlspecialchars($product['id']) ?></td>
                            <td><?= htmlspecialchars($product['name']) ?></td>
                            <td>$<?=
htmlspecialchars(number_format($product['price'], 2)) ?></td>
                            <td><?= htmlspecialchars($product['stock']) ?></td>
                            <td>
                                <a href="index.php?
controller=product&action=view&id=<?= $product['id'] ?>" class="btn btn-sm btn-
info">Ver</a>
                                <a href="index.php?
controller=product&action=edit&id=<?= $product['id'] ?>" class="btn btn-sm btn-
primary">Editar</a>
                                <a href="index.php?
```

```
controller=product&action=delete&id=<?= $product['id'] ?>" class="btn btn-sm
btn-danger">Eliminar</a>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>
        <?php endif; ?>
    </div>
</body>
</html>
```

Beneficios de MVC en Proyectos Reales

La implementación del patrón MVC ofrece numerosos beneficios en proyectos reales:

1. **Escalabilidad**: A medida que el proyecto crece, la separación clara facilita la adición de nuevas funcionalidades
2. **Mantenibilidad**: Es más fácil encontrar y corregir errores cuando el código está organizado lógicamente
3. **Seguridad**: Ayuda a implementar mejores prácticas de seguridad al procesar entradas del usuario en el controlador
4. **Reutilización**: Permite reutilizar componentes en diferentes partes de la aplicación
5. **Pruebas**: Facilita la implementación de pruebas unitarias al tener componentes separados

MVC y Frameworks PHP

El patrón MVC es la base de muchos frameworks PHP populares:

- **Laravel**: Implementa MVC de manera robusta con características adicionales
- **CodeIgniter**: Proporciona una implementación liviana de MVC
- **Symfony**: Ofrece componentes MVC flexibles
- **Yii**: Framework MVC de alto rendimiento

Comprender el patrón MVC te preparará para trabajar con cualquiera de estos frameworks, incluido Laravel, que es tu objetivo final.

Ejercicios Prácticos

Para consolidar el conocimiento sobre MVC, puedes realizar estos ejercicios:

1. Implementa un sistema CRUD para una entidad "Categoría" siguiendo el patrón MVC
2. Extiende el ejemplo de productos para incluir filtrado y paginación
3. Implementa un sistema de autenticación utilizando MVC
4. Crea un panel de administración para gestionar productos y categorías

Conclusión

El patrón Modelo Vista Controlador es fundamental para desarrollar aplicaciones PHP bien estructuradas. Dominar este patrón te proporcionará:

1. Un marco conceptual para organizar tu código
2. Una base sólida para aprender frameworks como Laravel
3. Habilidades para desarrollar aplicaciones web escalables y mantenibles
4. Una mejor comprensión de cómo funciona la arquitectura de software moderna

Al pasar de los conceptos algorítmicos básicos a este patrón de diseño, estás dando un paso importante hacia el desarrollo de aplicaciones web profesionales con PHP y, eventualmente, con Laravel.